Assignment 04 AI Evidence

Boranov Daniyar | Student ID 24B031718 | Thursday 14:00–17:00

This appendix records actual prompts, preserved responses and technical verification for the existing Smart Campus application. It is separate from the main engineering report. The original Assignment 03 AI evidence is unchanged.

Tool and review record

Tool: Codex. Model family: GPT-6, as identified by the session instructions; the exact deployment identifier was not exposed. Work date: 5 November 2026, Asia/Almaty.

The Task 1 plan was explicitly approved in the full request reproduced below. No other personal review or independently executed student tests are inferred from that approval. The reflection is a first-person draft prepared for final review.

Output used and technical decisions

The approved plan guided the four-room catalog, pure capacity/equipment filtering, selected-room state and real UI tests. The existing time and overlap validator and Python reference were preserved. One actual AI recommendation was modified: verification builds use an ignored output path because the deployment folder is tracked. Equipment ALL matching and touching endpoints were checked against the assignment and original contract.

Verification sources include the complete assignment PDF, production source, preserved baseline bundle and hashes, Git diffs, actual regression/UI logs and screenshots. Final source commit: 10eeb3812d6f9491cd47dcd72f5bf03b0884f88c.

Actual final command results

Python regression: 47/47 passed. Command: python3 -m unittest -v test_booking

Frontend rules and catalog/state: 76/76 passed. Command: npm test

ESLint: Exit 0. Command: npm run lint

TypeScript: Exit 0. Command: npm run typecheck

Production build: Exit 0. Command: npm run build -- --outDir output/a4-final-build

Chromium integration and UI: 22/22 passed. Command: A4_UI_RUN=final-local npm run test:ui -- --project=chromium

Anonymous public Chromium UI: 22/22 passed. Command: A4_BASE_URL=https://smart-campus-room-av-j4pb.bolt.host/ A4_UI_RUN=public-bolt-final npm run test:ui -- --project=chromium

Test correction and preserved failures

The first UI run passed 20 of 21 tests. A native-select keyboard test assumed an unreliable ArrowDown and Enter sequence. A focused check supported native type-ahead; the corrected check passed 1 of 1. The final suite passed 22 of 22. No application fix was required. Original failing logs and the corrected-run evidence are retained in evidence/implementation-testing.

Public deployment record

The A4 application was synchronized with the existing Bolt.new project and published at the original Bolt.host address. All 22 browser checks passed without signing in. The published JavaScript and CSS match the tested local production build byte for byte.

Task 1 exact user prompt

The following text is preserved verbatim. Layout wrapping does not change the historical exchange.

Read the attached Software Engineering Assignment #04 PDF completely and inspect my existing Assignment #03 repository: bdaniyar/boltnew3assSE.

Start with Task 1 — Codebase Analysis ONLY.

Before changing any code:

1. Record the current main commit and preserve a recoverable A4 baseline.
2. Create a concise repository map with real file and function references.
3. Identify at least three genuine maintainability risks supported by the existing source.
4. Explain how AC1–AC6 are currently implemented and how they will remain unchanged.
5. Propose a minimal feature extension for at least four fictional rooms and F1–F5 filtering.
6. Propose a refactoring structure separating room data, booking/availability rules, filtering logic, and UI.
7. Prepare an executable regression and integration/UI testing plan.
8. Identify possible AI recommendations that should be rejected or modified, with technical reasons.
9. List all evidence, diagrams, reports and source files required by Assignment #04.

Do not repeat the Practice #08 debugging exercise. Do not create a new SKILL.md. Do not create a new Bolt project. Do not implement changes yet.

Use the Assignment #04 PDF as the source of truth.

Show a concise prioritized plan with review decisions, then STOP for my approval. Preserve the actual prompt and full response for AI evidence.

Task 1 complete assistant response

The following text is preserved verbatim. Layout wrapping does not change the historical exchange.

Task 1 analysis is complete. I read all 30 pages of :codex-file-citation{path="/Users/daniarboranov/Downloads/Lesson 08 - Software Engineering - Assignment 04.pdf" purpose="source"}. Application code, tests and A3 evidence remain unchanged.

The full analysis contains the repository map, source references, AC traceability, proposed room data, executable testing plan and complete deliverable checklist:

[Read the Task 1 analysis](/Users/daniarboranov/Documents/code_materials/se/boltassignmnet/boltnew3assSE/A4_Boranov_Daniyar/Task1_Codebase_Analysis.md)

Current local and GitHub `main`: **`55cab2a186df4107ab7189f509c259006af28f3f`**. Preserved as local tag `a4-baseline-2026-11-05` and a [verified recovery bundle](/Users/daniarboranov/Documents/code_materials/se/boltassignmnet/boltnew3assSE/A4_Boranov_Daniyar/evidence/a4-baseline.bundle). Fresh baseline checks passed: **Python 47/47, frontend 37/37**, lint, type checking and build.

The main findings are:

| Source / responsibility | Maintainability risk |

|---|---|

| `src/App.tsx:27` — `handleCheck`; `:95` — booking display | Both use global `SAMPLE_BOOKINGS`; a room selector alone would use the wrong room's data. |

| `src/types.ts:8` — `Room`; `src/data/sampleData.ts:3` — fixture | Room lacks capacity, equipment and its own bookings. |

| `src/App.tsx:39–64` — input handlers | Five separate result-clearing locations make F5 easy to miss. |

| `scripts/frontend-test.mjs:17–49` — regression runner | Exercises rules but never renders the UI or changes selection. |

| `src/components/ResultPanel.tsx:11,27,49` — feedback | Three repeated status wrappers can drift during changes. |

`src/utils/availability.ts:12` — `checkAvailability` — already implements AC1–AC4. Preserve integer/day-range/future-time validation, exactly 120 minutes, blocked rejection and strict overlap comparisons that allow touching endpoints. Preserve AC5 through readonly/frozen bookings and nonmutation tests; preserve AC6 through existing responsive components, labels and clear status feedback. Keep fixed time **540** and the Python reference unchanged.

The proposed catalog keeps A-102 and adds B-201, C-303 and D-404, with capacities **4, 8, 12 and 20**, distinct equipment/bookings and one blocked room. Full fictional records are in the analysis. F1 uses `capacity >= attendees`; F2 requires **all** selected equipment; F3 retains blocked rejection; F4 shows a clear no-match state; F5 clears old results when switching rooms. Checks and booking displays use the same selected record.

| Priority | Proposed work after approval | Review decision / gate |

|---|---|---|

| P1 | Separate catalog/types and pure filtering from existing availability rules. | Preserve A-102 and every existing regression fixture. |

| P2 | Add filter/selector components; derive the chosen room from its ID. | Verify selected-room bookings, blocked status and combined filters. |

| P3 | Centralize result invalidation and share the feedback wrapper. | Verify empty state, room switching, original examples and responsive feedback. |

| P4 | Run regression/UI checks and prepare final evidence. | Record actual totals, diff review, diagrams, quality comparison and limitations. |

The test plan retains both suites and adds exact/insufficient capacity, ALL-equipment matching, room isolation, immutability, F4/F5 and real UI events. It covers 120/121 minutes and touching/overlapping intervals. Proposed Chromium UI testing uses [Playwright's documented runner](https://playwright.dev/docs/intro) and [local-server configuration](https://playwright.dev/docs/test-webserver); commands and proposed files are recorded, but no new tests or dependency were added.

One actual AI recommendation was modified: ordinary `npm run build` became a build with an ignored output directory because `dist/` is tracked. Reject inclusive overlap, ANY-equipment matching, live time, booking mutation and backend expansion because they conflict with the requirements.

Final submission needs initial/final source or commits, README commands, executable tests/results, an analysis/refactoring report, before/after diagrams, two design justifications, a measured quality table, limitations, a **200–300-word reflection**, and an AI appendix with genuine review evidence. The PDF does not specify mandatory filenames or a PDF/DOCX submission format.

Your [exact prompt](/Users/daniarboranov/Documents/code_materials/se/boltassignmnet/boltnew3assSE/A4_Boranov_Daniyar/evidence/ai-exchanges/01-user-prompt.txt) and [full response](/Users/daniarboranov/Documents/code_materials/se/boltassignmnet/boltnew3assSE/A4_Boranov_Daniyar/evidence/ai-exchanges/01-assistant-response.md) are preserved.

Stopped before implementation. Please approve or amend this plan.

Exact approval and full implementation request

The following text is preserved verbatim. Layout wrapping does not change the historical exchange.

Assignment #04 — Approved Plan and Full Implementation

I approve the proposed Task 1 analysis and implementation plan (P1–P4).

Please proceed with the full implementation of Software Engineering Assignment #04, using the attached 30-page assignment PDF as the source of truth.

Complete Tasks 2–6, preserve the existing Task 1 analysis, and prepare the complete submission package.

1. Student and project details

- Full name: Boranov Daniyar
- Student ID: 24B031718
- Practice class: Thursday 14:00–17:00
- Assignment: Software Engineering Assignment #04
- Project: Smart Campus Room Availability Checker
- English level: B1–B2

Existing repository:

https://github.com/bdaniyar/boltnew3assSE

Existing Bolt.new project:

https://bolt.new/~/sb1-bskzikuu

Previous A3 published demo:

https://smart-campus-room-av-j4pb.bolt.host/

Continue the existing application. Do not create a replacement application, another Bolt project, or an unrelated codebase.

Do not repeat the Practice #08 debugging exercise, and do not create another SKILL.md.

2. Preserve the A3 baseline

The approved Task 1 analysis has already recorded:

Baseline commit:

55cab2a186df4107ab7189f509c259006af28f3f

Baseline tag:

a4-baseline-2026-11-05

Recovery bundle:

A4_Boranov_Daniyar/evidence/a4-baseline.bundle

Before editing:

1. Verify that the recovery tag and bundle are valid.
2. Preserve the original A3 implementation.
3. Do not rewrite or delete the existing evidence.
4. Preserve the original test results: Python 47/47 and frontend 37/37.
5. Keep the previous AC1–AC6 implementation behavior unchanged.

6. Make all A4 development changes traceable through Git commits.

7. Preserve actual before/after source snapshots or commit references.

Do not modify the Python reference implementation unnecessarily.

3. New functional requirements — F1–F5

Extend the application to support at least FOUR fictional university rooms.

Use the approved room catalog from Task 1:

- A-102
- B-201
- C-303
- D-404

Use the approved fictional capacities, equipment, blocked states and booking intervals described in Task1_Codebase_Analysis.md.

Each room must have:

- room ID
- room name
- capacity
- equipment list
- blocked status
- its own existing bookings

Implement:

F1 — Capacity filtering

A room matches only if:

room.capacity >= requestedAttendees

Test exact capacity, insufficient capacity and larger capacity.

F2 — Equipment filtering

A room matches only when it contains ALL selected required equipment.

Do not use ANY-equipment matching.

Selecting no equipment should not exclude a room based on equipment.

F3 — Blocked rooms

A blocked room must NEVER be reported as Available.

It may remain visible in the filtered catalog if it satisfies the capacity/equipment requirements, but it must show its blocked status clearly.

F4 — Empty state

When no rooms satisfy the selected requirements, display a clear message explaining that no matching rooms were found.

Do not show stale availability results.

F5 — Room switching

When the selected room changes:

- clear or correctly refresh the previous availability result;
- use the new room's blocked status;
- use the new room's existing bookings;
- never accidentally use another room's bookings.

Additional important condition:

If filtering changes and the previously selected room is no longer included in the matching results, clear that selection and the previous availability result.

Do not continue checking a room that is no longer selectable.

4. Preserve AC1–AC6

The original acceptance criteria must remain valid:

AC1:

0 <= start < end <= 1440, with integer times and start > now.

AC2:

Maximum duration 120 minutes, including exactly 120.

AC3:

Blocked rooms are unavailable.

AC4:

Reject overlapping bookings but permit touching endpoints.

AC5:

Only check availability. Never create, edit, delete, confirm or mutate bookings.

AC6:

Clear validation feedback and a responsive mobile layout.

Keep the fixed fictional demo time:

now = 540 (09:00)

Keep all existing regression tests.

Do not introduce:

- authentication
- database
- backend
- external APIs
- payments
- actual booking creation
- unrelated functionality

5. Task 2 — Feature extension

Implement the new room catalog and filtering interface.

The final website should allow users to:

1. Enter the required attendee count.
2. Select zero or more required equipment items.
3. View only matching rooms.
4. Select a matching room.
5. See its capacity, equipment and blocked status.
6. View that room's fictional existing bookings.
7. Enter a requested time interval.
8. Check availability using ONLY the selected room's data.
9. Receive clear Available / Unavailable feedback.
10. Understand when no rooms match the filters.

Maintain the visual style of the existing Smart Campus application.

Do not perform a major redesign.

Keep desktop and mobile layouts readable and usable.

6. Task 3 — Refactoring

Refactor the application without changing the original behavior.

Separate responsibilities for:

- room catalog and fictional data
- domain types
- availability and booking validation
- capacity/equipment filtering
- selected-room state
- UI presentation and feedback

Address the genuine maintainability risks from Task 1:

R1. Global SAMPLE_BOOKINGS usage in App.tsx.

R2. Room type and fixture lacking the new fields.

R3. Multiple result-clearing handlers.

R4. Lack of integration/UI event coverage.

R5. Duplicated status presentation where appropriate.

Avoid unnecessary abstractions, overengineering or a full framework rewrite.

For every meaningful refactoring step:

1. Explain the targeted issue.
2. Record the source change.
3. Run appropriate tests.
4. Inspect the diff.

5. Preserve the result.

Create TWO architecture diagrams:

- BEFORE — original A3 module/component structure.
- AFTER — final A4 module/component structure.

The diagrams must reflect the real source code, not an imaginary architecture.

Use Mermaid, PlantUML or another clear format and export readable images for the report.

Explain at least TWO important design decisions and their trade-offs.

7. Task 4 — Automated testing

Create and execute at least TEN automated tests.

The final suite should substantially exceed the minimum where useful.

Preserve and run the existing:
- Python regression suite
- frontend logic suite
- lint
- type checking
- production build

Add new tests for:

- F1 exact capacity
- F1 insufficient capacity
- F2 all required equipment
- F2 missing one required equipment item
- F2 no equipment selected
- F3 blocked room
- F4 zero matching rooms
- F5 switching rooms
- F5 changing filters invalidates a selection
- correct selected-room bookings
- room isolation
- immutable booking inputs
- multiple bookings
- 120-minute boundary
- 121-minute rejection
- touching endpoints
- overlapping intervals
- invalid time
- narrow/mobile layout

Include at least ONE real integration or browser/UI behavior test.

Prefer Playwright if it can be installed and run reliably.

If browser automation is unavailable, use another real executable approach and describe its limitations accurately.

Do not fabricate test results.

Save:

- actual commands
- stdout/stderr or test logs
- actual pass/fail counts
- exit codes
- relevant screenshots
- UI verification evidence

Check the application after each meaningful change and run the complete regression suite at the end.

8. Task 5 — Quality evaluation

Create a measured comparison of the ORIGINAL A3 baseline and FINAL A4 version.

Include a table covering:

1. Automated tests passed / total.
2. Number of duplicated logic locations removed.
3. Module/component structure and responsibilities.
4. At least one remaining limitation.

Explain exactly how every metric was obtained.

Do not invent numbers or describe subjective impressions as measurements.

Make the distinction between adding a new feature and improving maintainability clear.

Include actual source/file references.

9. Task 6 — AI reflection

Prepare a 200–300-word reflection in natural B1–B2 English.

Address:

- What context was provided to the AI assistant?
- Which AI suggestions were useful?
- Which recommendations were incorrect, rejected or modified?
- What was actually verified?
- How did the tests influence engineering decisions?
- What would be improved next?

Use genuine decisions from this workflow, including the approved Task 1 plan.

Do not invent my personal reviews or actions.

Prepare a readable first-person draft for my final review.

10. AI evidence and human decisions

Preserve the actual Task 1 prompt and complete response already saved under:

A4_Boranov_Daniyar/evidence/ai-exchanges/

Record AI usage accurately, including:

- assistant/tool name
- date
- prompt summary or selected exact prompts
- output used
- problems identified
- changes made
- verification sources
- accepted/rejected/modified recommendations

Preserve genuine exchanges and technical evidence.

Do not fabricate prompts, screenshots, independent reviews or approvals.

Document at least one real AI recommendation that was rejected or modified, with a technical reason.

If you encounter another meaningful optional design choice requiring my approval, recommend a default and identify it clearly. Continue with the already approved scope without unnecessary interruptions.

11. Important report writing rules

The main report must look like a normal university Software Engineering assignment.

Use natural, simple B1–B2 English.

Do NOT fill the report with unnecessary administrative commentary such as:

- "The student delegated this task to AI."
- "The student has not confirmed..."
- "AI generated this report."
- "Student decision pending."
- "The root agent performed..."
- "Human approval not recorded."
- "This section was written by AI."
- "The student did not personally execute..."

Use normal engineering language:

- "The original application was preserved."
- "The new catalog supports four rooms."
- "The filtering logic was extracted into a separate module."
- "The automated tests passed."
- "The following limitations remain."

Preserve honest descriptions of actual verification and mandatory AI-use disclosure in the dedicated evidence appendix.

Do not invent personal actions or conceal AI involvement where the assignment requires it.

Keep the report concise, professional and readable.

Avoid duplicate paragraphs and unnecessarily long source dumps.

12. Report and documentation

Create the following deliverables:

A4_Boranov_Daniyar_Report.docx
A4_Boranov_Daniyar_Report.pdf
AI_Evidence.pdf
README.md

Also preserve:

- initial and final source or Git commit references
- test source files
- actual test logs
- architecture diagrams
- screenshots
- quality comparison evidence
- baseline recovery information

The report must contain:

1. Assignment title and student information.
2. Project overview.
3. Existing A3 baseline.
4. Codebase map and three or more maintainability risks.
5. Prioritized improvement plan.
6. AC1–AC6 preservation.
7. F1–F5 new requirements and implementation.
8. Before/after module diagrams.
9. At least two refactoring/design decisions.
10. Automated regression and integration testing.
11. Test results.
12. Quality evaluation table.
13. Evidence and screenshots.
14. Remaining limitations.
15. AI reflection (200–300 words).
16. AI-use indication and references.

Include:
Boranov Daniyar
Student ID: 24B031718
Thursday 14:00–17:00

Make the DOCX fully editable.

Keep tables, headings and screenshots professional and readable.

Render the final PDF and visually inspect all pages.

Do not crop images, overlap text or split tables badly.

13. Bolt.new and public deployment

The existing A3 Bolt.new project is:

https://bolt.new/~/sb1-bskzikuu

The previous public demo is:

https://smart-campus-room-av-j4pb.bolt.host/

After completing the A4 implementation:

1. Commit and push the A4 source.
2. Verify that the original A3 baseline remains recoverable.
3. Check whether the Bolt.new project has synchronized with the latest source.
4. Publish/update the Bolt-hosted site if the environment allows it.
5. Verify the final live site without signing in.
6. Verify the new four-room catalog.
7. Verify the capacity filter.
8. Verify the equipment filter.
9. Verify the empty state.
10. Verify room switching and room-specific availability.
11. Verify that the original time validation still works.

Do not assume GitHub commits automatically update Bolt.host.

If Bolt.new publishing requires my manual action, give the exact instructions and preserve every other completed deliverable.

IMPORTANT:

Include the actual final working public URL and Bolt.new project URL as clickable links near the beginning of the final PDF.

Do not leave a stale A3 link presented as a verified A4 deployment.

Do not invent a new URL.

14. Final submission package

Create an organized A4_Boranov_Daniyar folder containing:

- A4_Boranov_Daniyar_Report.pdf
- A4_Boranov_Daniyar_Report.docx
- AI_Evidence.pdf
- README.md
- diagrams/
- tests/
- evidence/
- source references or source snapshots as appropriate

Include the source code needed to reproduce the result or accessible initial/final Git commit references.

Also create:

A4_Boranov_Daniyar.zip

The ZIP must include the actual required evidence and not only the PDF documents.

Make README.md explain:

- the application
- technologies
- setup instructions
- run instructions
- testing commands
- source locations
- baseline recovery
- final live demo URL
- Bolt project URL

Do not include credentials, API keys, .env secrets, node_modules or unnecessary build cache files.

15. Final audit

Before finishing, compare the deliverables against EVERY requirement in the Assignment #04 PDF.

Confirm:

- at least four fictional rooms
- each room has ID/name/capacity/equipment/blocked/bookings
- F1–F5 implemented
- AC1–AC6 preserved
- at least three real maintainability risks documented
- two or more design decisions explained
- before/after architecture diagrams included
- at least ten executable tests
- at least one integration/UI check
- actual execution logs included
- measured quality evaluation
- remaining limitation documented
- 200–300-word reflection
- AI evidence and actual review decisions
- baseline and final code recoverable
- README and reproducible commands
- DOCX and PDF formatting checked
- actual public demo links included
- all required files present in ZIP

The assignment deadline is November 8, 2026 at 23:59:59.

Formatting and completeness are critical because the assignment warns that incorrect or incomplete submissions may be rejected without resubmission.

16. Execution

You have approval to perform the proposed P1–P4 implementation.

Proceed through Tasks 2–6 autonomously.

Do not stop after writing code or running the first test suite.

Complete all possible implementation, tests, documentation and final deliverables in this execution.

Do not perform unrelated repository cleanup.

Do not fabricate verification.

If an external action is unavailable, complete everything else and state the exact remaining action.

At the end provide:

1. Summary of completed Tasks 1–6.
2. Real test results and execution commands.
3. Main source changes and final commit.
4. Any remaining blockers.
5. Actual Bolt.new and public demo URLs.
6. Downloadable DOCX, PDF and final ZIP.
7. A short final submission checklist.

The objective is a fully implemented, tested, documented and submission-ready Software Engineering Assignment #04.

Review of the Bolt synchronization recommendation

Tool: Bolt.new Standard agent; the exact model was not identified. The request concerned synchronization of the existing project, without code generation or publication. The complete visible response was preserved, including progress labels; hidden reasoning and unexpanded tool details are not claimed.

The Publish control later generated the exact publication request preserved below. The visible publication response and independent anonymous HTTP, build-asset and browser checks verify the actual result; a chat success statement alone is not treated as deployment proof.

The response treated unavailable shell Git commands as a synchronization blocker. That limitation was not accepted as proof that the built-in GitHub interface or publication was unavailable. The actual Code view and existing interface were checked separately. This modified recommendation and its verification sources are recorded in 03-bolt-sync-observation.json.

After publication, a managed repository synchronization removed an ignored tracked baseline log and deployment assets. The historical log was restored byte for byte, the verified A4 build was tracked, and narrow ignore exceptions protected these records. Production TypeScript, Python and test source remained identical to the tested version. The exact artifact-only change and preservation checks are recorded in evidence/refactoring/bolt-sync-preservation-fix.json.

Existing Bolt project exact synchronization prompt

Synchronize this EXISTING Bolt project with its already connected GitHub repository bdaniyar/boltnew3assSE, branch main. The tested Assignment 04 implementation is commit 4b2631f8519cb62321314d8c4b77c0d909c056f7 (four rooms A-102/B-201/C-303/D-404, capacity and ALL-equipment filters). The current Code view still shows the old useState/SAMPLE_BOOKINGS App.tsx. Pull the current GitHub main into this project and keep its exact tested source. Do not generate, rewrite or refactor code; do not create a new project, modify preserved evidence, or change history/tags. If GitHub synchronization is unavailable, explain the precise blocker and stop. After a successful pull, run the production build and report what source was loaded. Do not publish yet.

Existing Bolt project complete visible agent response

Thoughts

I'll investigate the current git state and the repository configuration to determine whether I can synchronize with GitHub.

0

1

2

3

4

5

6

7

8

9

commands executed

Let me verify whether any git tooling is accessible at all, and check what the current Code view shows versus the tested state on disk.

0

1

2

3

4

5

6

7

8

9

actions taken

I need to stop here and report a hard blocker.

GitHub synchronization is unavailable — exact blocker

Git is not available in this Bolt workspace. Every git command — git status, git remote, git log, even which git — is blocked by the environment with this message:

"Blocked: git. This workspace is not a git repository and has no GitHub remote. Bolt saves your file changes automatically. Any repository, branch, or commit made here is deleted at the end of the turn, so its branch names and commit SHAs never reach GitHub."

There is no .git directory at /, and the .bolt/config.json only records the Vite template — there is no GitHub remote configured here. Bolt's workspace is a sandboxed file tree, not a git working copy, so I cannot pull, fetch, clone, or otherwise reach bdaniyar/boltnew3assSE branch main at commit 4b2631f.

I did not generate, rewrite, or refactor any code, and I did not modify any preserved evidence.

What I can report about the source currently on disk

For transparency, the files already present at src/ are not the old useState/SAMPLE_BOOKINGS version the Code view shows. They match the Assignment 04 implementation you described:

src/App.tsx uses a useReducer + checkerReducer pattern (not the old useState), imports RoomFilters, RoomSelector, RoomCard, and reads from src/data/rooms and src/state/checker.

New files consistent with the four-room / capacity / equipment-filter design exist: src/components/RoomFilters.tsx, src/components/RoomSelector.tsx, src/data/rooms.ts, src/state/checker.ts, src/utils/filtering.ts, src/data/exampleCases.ts.

In other words, the working tree on disk already appears to be the tested Assignment 04 source — the stale " Code view" you see is a Bolt preview/cache artifact, not the actual files. But because I cannot run git, I cannot verify that these files byte-for-byte match commit 4b2631f, and I cannot pull to guarantee that match.

What this means and the next step

Because git synchronization is the core of your request and it is genuinely blocked, I'm stopping here as you instructed rather than attempting a workaround that wouldn't be a real sync.

To get the Bolt project aligned with GitHub bdaniyar/boltnew3assSE@4b2631f, you would need to either:

Connect/sync the GitHub repo through Bolt's built-in GitHub integration (via the Bolt UI, not something I can invoke from here), or

Push the current on-disk source to that repo from a machine with git access, after confirming it matches the tested commit.

I did not run the production build, because you asked for it only after a successful pull — and no pull happened. Would you like me to run npm run build against the source currently on disk anyway, clearly labeled as "on-disk source, not GitHub-verified"?

Existing Bolt publication UI generated request

Publish this application.

Existing Bolt publication complete visible response

Your site has been successfully published! You can view it at:

https://smart-campus-room-av-j4pb.bolt.host

To add a custom domain or purchase one through Bolt, upgrade to Bolt Pro